Expressões regulares em JavaScript: relatório técnico

Equipe 2 · JavaScript / Node.js · Thiago Corobinski Carmona
Linguagens Formais e Autômatos · Prof. Luciano Cardoso · 2º semestre de 2026 · Trabalho prático "Regex Multilinguagem"
Código: github.com/ThiagoCCarmona/regex-js · Apresentação: regex-js.tccodes.com.br

1. Objetivo e ambiente

Reproduzimos em JavaScript as seis atividades do laboratório de expressões regulares feito em Python: reconhecimento, validação, extração com grupos, substituição e divisão, parser de logs e analisador léxico. O código roda em **Node.js 24.21** (compatível com Node ≥ 20), usa somente a classe nativa `RegExp` e os métodos de `String` e **não tem dependências externas**. Os testes usam o executor nativo `node:test`.

A entrega fica em `equipe_02_javascript/`, com um módulo por etapa em `src/`, testes em `testes/` e arquivos de entrada em `dados/`. O comando `npm test` executa **49 testes**, entre positivos, negativos e de borda. O comando `npm run demo` imprime, para cada caso do enunciado, a saída **obtida** ao lado da **esperada** (**47/47 verificações ok**).

Dados reconstruídos. O enunciado cita `dados/logs.txt` e o texto-base do laboratório Python, mas esses arquivos não estavam disponíveis para a equipe. Por isso os reconstruímos de modo que reproduzam exatamente os resultados esperados do enunciado: 6 linhas de log, 5 válidas e 1 inválida, com INFO=3, WARN=1 e ERROR=1, ana=2, bruno=2 e carla=1, e 5 ações distintas; o texto-base tem os 2 e-mails, as 2 datas e as 3 hashtags citados. O formato de log adotado é `DD/MM/AAAA HH:MM:SS NIVEL usuario IP acao`.

2. Resultados da matriz mínima de testes

Módulo	Entrada	Critério do enunciado	Obtido
Reconhecimento	5 identificadores e a frase com números	Classificação exata e [2026, 32, 6]	**total, _contador e nome_completo ✓; 2valor e x-y ✗; ['2026','32','6']**
Validação	3 matrículas, 3 datas e 2 telefones	Aceitar somente os formatos especificados	**CC-2026-001 ✓; SI-2026-001 e CC-26-001 ✗; 99/99/9999 com sintaxe ✓ e semântica ✗**
Captura	2 e-mails, 2 datas e 3 hashtags	Valores e grupos corretos	**datas com groups {dia, mes, ano}; 08:30 e 23:59 ✓; 25:61 ✗**
Transformação	Espaços, máscara de CPF e divisão	Saídas idênticas ao enunciado	**'LFA com Python é prática!'; '123.***.***-00'; [LFA, Python, Regex, Automatos]**
Logs	Arquivo compartilhado (6 linhas)	5 válidas, 1 inválida e contagens corretas	**5/1; ampliado 7/2; IP 300.1.1.1, 31/02 e 25:00 rejeitados na semântica**
Tokens	Expressões do enunciado e o símbolo @	Tipos, lexemas, ordem e erro explícito	**sequências idênticas; "Símbolo inesperado '@' na posição 6 (coluna 7)"**

3. Respostas às perguntas orientadoras

3.1 Quais funções equivalem a reconhecimento completo, busca, extração global, substituição e split?

Operação	Python (`re`)	JavaScript
Reconhecimento completo	`re.fullmatch(p, s)`	`/^p$/.test(s)`: não existe *fullmatch*; usamos âncoras
Busca (1ª ocorrência)	`re.search(p, s)`	`/p/.exec(s)` ou `s.match(/p/)`, sem a flag `g`
Extração global	`re.findall` / `re.finditer`	`s.match(/p/g)` (só os textos) / `s.matchAll(/p/g)` (com grupos e índice)
Substituição	`re.sub(p, r, s)`	`s.replace(/p/g, r)` ou `s.replaceAll(/p/g, r)`
Divisão	`re.split(p, s)`	`s.split(/p/)`
Compilação	`re.compile(p)`	literal `/p/` (compilado na carga do módulo) ou `new RegExp(p)`

3.2 Quais diferenças de sintaxe e comportamento observamos em relação ao `re` do Python?

- **Grupos nomeados:** `(?<nome>…)` em JS e `(?P<nome>…)` em Python. A retro-referência é `\k<nome>` (Python: `(?P=nome)`) e, na substituição, `$<nome>` e `$1` (Python: `\g<nome>` e `\1`).
- **Classes abreviadas:** em JS, `\d`, `\w` e `\b` são sempre ASCII. Em Python 3, com padrões *str*, são Unicode por padrão (`\d` casa "٣"). Por isso usamos classes explícitas, como pede o enunciado.
- **`$` e quebra de linha final:** em Python, `re.match(r'^\d+$', '123\n')` casa, porque `$` aceita um `\n` final. Em JS, `/^[0-9]+$/.test('123\n')` é `false`, então nossos validadores são estritos sem precisar de `\Z`, que não existe em JS.
- **Estado na regex:** com as flags `g` ou `y`, o objeto guarda `lastIndex` entre chamadas; não há equivalente em Python (seção 5).
- **Exigência da flag g:** `matchAll` e `replaceAll` lançam `TypeError` se a regex não tiver `g`; `replace` sem `g` troca só a primeira ocorrência (o `re.sub` troca todas).
- **Flags próprias:** `y` (sticky, usada no léxico), `d` (índices dos grupos), `u` e `v` (Unicode e `\p{…}`). Os modificadores embutidos do tipo `(?i:…)` só chegaram no ECMAScript 2025.

3.3 Como o programa distingue formato sintaticamente válido de informação semanticamente válida?

Em duas camadas. A regex responde se a cadeia tem a **forma** certa: `^(?<dia>[0-9]{2})\/(?<mes>[0-9]{2})\/(?<ano>[0-9]{4})$` aceita `99/99/9999`. Em seguida, os grupos são convertidos em números e conferidos por código comum (mês entre 1 e 12, dias por mês e ano bissexto). `validarData` devolve `{ sintatica, semantica, motivo }`, e por isso `99/99/9999` resulta em `{ true, false, 'mês 99 não existe' }`. O parser de logs faz o mesmo: `300.1.1.1` e `31/02/2026` passam pela regex e são rejeitados por `validarSemantica`, com o motivo. A regex poderia codificar algumas faixas, como fizemos nos horários (`[01][0-9]|2[0-3]`), mas regras como bissexto ficariam ilegíveis numa regex. Essa separação também mostra o limite do que uma linguagem regular expressa de forma prática.

3.4 Qual é o papel dos grupos de captura e como são acessados?

Os grupos delimitam partes do casamento para extraí-las ou reutilizá-las. Em JS, o resultado de `exec`/`matchAll` é um array: `m[0]` é o casamento inteiro, `m[1]…m[n]` são os grupos posicionais e `m.groups` é um objeto com os nomeados. A Etapa 3 decompõe a mesma data das duas formas (`decomporDataPosicional` usa `m[1]`; `decomporDataNomeada` usa `m.groups.dia`), e um teste garante que os resultados são iguais. Os nomes deixam o código autoexplicativo e resistem a mudanças: incluir um grupo novo não desloca os demais. Na Etapa 5, os seis campos do log saem direto de `{ ...m.groups }`.

3.5 Como o tokenizador decide entre operadores com prefixos comuns (= e ==)?

O tokenizador usa uma regex mestre, `(?<NUMBER>…)|(?<ID>…)|(?<RELOP>==|!=|<=|>=|<|>)|(?<ASSIGN>=)|…|(?<MISMATCH>[\s\S])`, com a flag `y`. O motor do JS testa as alternativas **da esquerda para a direita e fica com a**

primeira que casa; não procura o casamento mais longo, como faria um analisador léxico gerado a partir de um AFD. Por isso a ordem é a regra de prioridade: `RELOP` vem antes de `ASSIGN`, e dentro de `RELOP` os operadores de dois caracteres vêm antes de `<` e `>`. A flag sticky obriga cada casamento a começar exatamente em `lastIndex`, de modo que nenhum caractere é pulado: o que não casa com nada cai em `MISMATCH` e vira erro com posição e coluna.

3.6 Quais limitações do motor são relevantes para este laboratório?

- **Backtracking e ReDoS:** o V8 (Irregexp) usa backtracking. `/(a+)+$/` contra `"a…a!"` leva tempo exponencial: medimos 2, 12, 178 e 731 ms para 16, 20, 24 e 26 caracteres, e mais de 2 s para 28. Não há quantificadores possessivos nem grupos atômicos para conter isso, e `test()` não pode ser interrompido. Por isso, no site, toda regex digitada por usuários roda num *worker* com limite de tempo.
- **ASCII em `\w` e `\b`:** palavras acentuadas exigem `\p{L}` com a flag `u` e um *lookbehind* no lugar de `\b`.
- **Sem `fullmatch`, `\A` ou `\Z`:** o reconhecimento completo depende de `^…$` sem a flag `m`.
- **`lastIndex` mutável:** compartilhar uma regex global entre chamadas causa resultados alternados.

3.7 Em que sentido as ER de bibliotecas e as ER da Teoria da Computação são semelhantes e diferentes?

Semelhanças: as duas partem das mesmas operações: união (`|`), concatenação, fecho de Kleene (`*`) e a cadeia vazia λ, que na biblioteca aparece como o casamento vazio (por exemplo, `/a*/` casa com `''`). Restrita a essas operações, e às derivadas `+`, `?`, `{m,n}` e classes, uma regex de JS descreve uma linguagem regular, com AFD equivalente. É o caso de `/^(a|b)*abb$/`.

Diferenças: as bibliotecas acrescentam recursos que **saem da classe das linguagens regulares**. Com retro-referência, `/^(?<p>[ab]+)\k<p>$/` reconhece L = { ww : w ∈ {a,b}$^+$ }, que não é regular (lema do bombeamento). *Lookahead*, *lookbehind*, âncoras e grupos de captura também não têm contrapartida na ER formal: uma ER define um conjunto de cadeias, enquanto a regex de biblioteca é um **procedimento de busca** que informa onde casou e o que capturou. Para sustentar esses recursos, o motor usa backtracking em vez de simular um AFD, e é daí que vem o risco de tempo exponencial. Motores como o RE2 do Go escolheram o caminho oposto: tempo linear garantido, sem retro-referências.

4. Problemas encontrados e como foram resolvidos

4.1 “laboratórios” cortado em “laborat”

Portado literalmente do Python, o padrão de `palavrasComL` seria `/\b[Ll]\w*/g`, que em JS devolve `['LFA', 'laborat']`: o `\w` para no `ó`. Trocamos por `/(?<![\p{L}\p{N}_])[Ll][\p{L}\p{N}_]*/gu`, com classes Unicode e a flag `u`. O *lookbehind* negativo substitui o `\b`, que também é ASCII. Um teste fixa o resultado em `['LFA', 'laboratórios']`.

4.2 Substituição “literal” que não era literal

No desafio da Etapa 4, `'preço: $5'.replaceAll('$5', '$&$&')` produziu `'preço: $5$5'`: mesmo com o padrão em string, o JS interpreta `$&`, `$1` e `$$` na string de substituição. A solução foi passar uma **função** como substituta (`replaceAll(alvo, () => novo)`), cujo retorno é inserido sem interpretação. O caso ficou registrado em teste.

4.3 `test()` alternando `true` e `false`

Ao reutilizar uma regex com a flag `g` em `test()`, chamadas seguidas com a mesma entrada retornavam `true`, `false`, `true`…, porque `lastIndex` avança após cada casamento. Corrigimos assim: validadores (`test`) usam regex **sem** `g`; a extração usa `match`/`matchAll`, que reiniciam o estado; e o tokenizador cria uma cópia (`new RegExp(RE_MESTRE)`) a cada chamada. Há um teste que chama `ehIdentificador` três vezes seguidas.

4.4 Prioridade de operadores e quebra de linha no léxico

Com `ASSIGN` antes de `RELOP`, `x==y` virava `ID ASSIGN ASSIGN ID`. Reordenar a especificação resolveu, e um teste fixa `x==y` como `ID RELOP ID`. Também trocamos `MISMATCH` de `.` para `[\s\S]`: o ponto não casa `\n`, e uma quebra de linha na entrada faria o laço com a flag sticky parar sem reportar erro.

5. Conclusão

As seis etapas reproduzem os resultados do laboratório Python com a `RegExp` nativa. As diferenças mais relevantes não estão na sintaxe básica, que é quase idêntica, mas no comportamento: a ausência de `fullmatch`, o `\w` só ASCII, o estado `lastIndex` e a interpretação de `$` na substituição. O trabalho também deixou concreta a distância entre a ER da teoria, um conjunto de cadeias com AFD equivalente, e a regex de biblioteca, um procedimento de busca mais expressivo e, por isso mesmo, sujeito a custo exponencial. As mesmas funções da entrega alimentam o site da apresentação, com demonstrações ao vivo e dois jogos.

Uso de IA generativa: o código, os testes e este texto foram produzidos com apoio do Claude (Anthropic), via Claude Code, e revisados e testados pelo autor; detalhes em `uso_ia.md`. **Referências:** MDN Web Docs, "Regular expressions" (developer.mozilla.org/docs/Web/JavaScript/Guide/Regular_expressions); ECMAScript 2025 Language Specification, §22.2 RegExp Objects; documentação do módulo `re` do Python 3; HOPCROFT, MOTWANI e ULLMAN, *Introdução à Teoria de Autômatos, Linguagens e Computação*.